A Report on

DAILY HEALTH & WELLNESS MONITOR

(A Machine Learning based Lifestyle Wellness Indicator using Random Forest, FastAPI and SQLite)

Submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

In

Computer Science & Engineering

By

MD SHAHBAZ ANSARI



AJAY KUMAR GARG ENGINEERING COLLEGE, GHAZIABAD

Dr. A P J ABDUL KALAM TECHNICAL UNIVERSITY
LUCKNOW
2026

CERTIFICATE

Certified that **Md Shahbaz Ansari** have carried,
out the project work presented in this report entitled **“Daily Health & Wellness Monitor”** for the award of **Bachelor of Technology** from Ajay Kumar Garg Engineering College, Ghaziabad, under my supervision. The report embodies result of original work and studies carried out by the students themselves and the contents of the report do not form the basis for the award of any other degree to the candidates or to anybody else.

Date: 06/10/2026

DECLARATION

We hereby declare that this submission is our own work and that, to the best of our knowledge and belief, it contains no material previously published or written by another person nor material which to a substantial extent has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgment has been made in the text.

NAME:MD SHAHBAZ ANSARI

Date: 06/10/2026

ABSTRACT

*The **Daily Health & Wellness Monitor** is a locally-run, machine-learning based web application that converts a person's everyday lifestyle data into an easy-to-understand wellness indicator. Every day the user enters eleven values – age, sleep hours, steps, exercise minutes, water intake, stress level, resting heart rate, calories consumed, screen time, weight and mood – on a single dashboard page. The system classifies the day as **Good**, **Moderate** or **Needs Improvement** using a trained **Random Forest classifier**, computes a separate and fully transparent **0–100 wellness score**, stores the entry permanently and plots the user's trend over time.*

*The application follows a clean layered architecture. The frontend is built with HTML, CSS, vanilla JavaScript and **Chart.js**; the backend is a **FastAPI** service with **Pydantic** request validation; a dedicated ML prediction layer loads a serialized scikit-learn **Pipeline (StandardScaler + RandomForestClassifier)** saved with joblib; and all submissions are stored in a **SQLite** database through **SQLAlchemy**. Model training is a separate offline process. Because public health datasets are mostly clinical and diagnosis-oriented, a synthetic lifestyle dataset of 4,000 records is generated, with Gaussian noise and random label flips added so that the model learns a realistic, imperfect decision boundary instead of memorising a formula.*

*On the held-out test set of 800 records the model achieved an accuracy of 67.87% and a macro F1-score of 0.616, with steps and exercise minutes emerging as the most important features. The project demonstrates an end-to-end ML product – data generation, training, evaluation, model serving, persistence and visualisation – while clearly stating that it is a lifestyle awareness tool and **not a medical device**.*

LIST OF FIGURES

Figure 1.	System Architecture of the Daily Health & Wellness Monitor	XV
Figure 2.	Offline Dataset Generation and Model Training Pipeline	XV
Figure 3.	Level 0 DFD (Context Diagram)	XVI
Figure 4.	Level 1 DFD (Expanded View)	XVII
Figure 5.	Level 2 DFD of the Prediction Process (2.0)	XVIII
Figure 6.	Flowchart of the Daily Entry Analysis	XIX
Figure 7.	Use Case Diagram	XX
Figure 8.	Sequence Diagram for POST /predict and GET /history	XXI
Figure 9.	ER Diagram of the health_records Table	XXI
Figure 10.	Class Distribution of the Synthetic Dataset (4,000 rows)	XXIV
Figure 11.	Dashboard Home Page (Before Analysis)	XXVIII
Figure 12.	Daily Entry Form Filled with Today's Values	XXIX
Figure 13.	Result Card Showing "Good" Status (Wellness Score 74/100)	XXIX
Figure 14.	Result Card Showing "Moderate" Status (Wellness Score 59/100)	XXIX
Figure 15.	Wellness Over Time Trend Chart	XXX
Figure 16.	Confusion Matrix of the Random Forest on the Test Set	XXXI
Figure 17.	Feature Importances of the Trained Random Forest	XXXI

TABLE of CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	CERTIFICATE	**II**
	ACKNOWLEDGEMENT	**III**
	DECLARATION	**IV**
	ABSTRACT	**V**
	LIST OF FIGURES	**VI**
1.	**INTRODUCTION**	**VIII**
	1.1 Problem Definition	VIII
	1.2 Background about the Project Idea	VIII
	1.3 Feasibility Study, Need and Significance	IX
	1.4 Novelty of Project	IX
	1.5 Technical Specification	X
2.	**AIM AND OBJECTIVE**	**XI**
3.	**SCOPE OF PROJECT**	**XII**
	3.1 Functional Scope	XII
	3.2 Technical Scope	XII
	3.3 Application Scope	XIII
	3.4 Out-of-Scope Areas	XIII
	3.5 Summary of the Project Scope	XIII
4.	**SYSTEM ANALYSIS & DESIGN**	**XIV**
	4.1 System Analysis	XIV
	4.2 System Architecture	XV
	4.3 Data Flow Diagrams (DFDs) of the Project	XV
	4.4 Flowchart	XVIII
	4.5 Use Case Diagram	XIX
	4.6 Sequence Diagram	XX
	4.7 ER Diagram	XXI
	4.8 Summary of System Design	XXII
5.	**IMPLEMENTATION**	**XXIII**
	5.1 Tables – Fields and Data Types of the Database	XXIII
	5.2 Used Algorithms / Approaches	XXIV
	5.3 Implementation of Modules	XXV
	5.4 Execution of the Project	XXVII
6.	**RESULTS / OUTPUTS & TESTING**	**XXVIII**
	6.1 All User Interfaces and Output Screens	XXVIII
	6.2 API Outputs	XXX
	6.3 Model Evaluation Results	XXX
	6.4 Design and Test Steps / Criteria	XXXII
	6.5 Testing Process	XXXIII
7.	**CONCLUSION AND FUTURE WORK**	**XXXV**
	7.1 Conclusion	XXXV
	7.2 Future Work	XXXV
8.	**REFERENCES**	**XXXVII**

CHAPTER 1
INTRODUCTION

1.1 Problem Definition

Modern lifestyles – long hours in front of screens, irregular sleep, low physical activity, inadequate water intake and constant stress – slowly affect a person's overall well-being. Most people do not notice these patterns because the information is scattered: steps live in a phone pedometer, sleep is a vague memory, and stress or mood is never written down at all. Even when people do note these numbers, raw values such as "6,200 steps" or "5.5 hours of sleep" are difficult to interpret as a whole, and there is no simple way to see whether the overall pattern of a day was healthy or not.

Existing fitness applications usually focus on a single metric (for example step count), are tied to a particular wearable device, require cloud accounts, or present medical-looking scores without explaining how they were produced. Clinical "health prediction" systems, on the other hand, are designed for diagnosis and depend on medical measurements that an ordinary user does not have.

The problem addressed by this project is therefore: **to build a simple, transparent and fully local system that takes a person's daily lifestyle inputs, uses a trained machine-learning model to classify the day into an understandable wellness category, computes an explainable wellness score, and keeps a history so that the user can observe trends over time – without claiming to diagnose any disease.** The **Daily Health & Wellness Monitor** is the solution developed to address this problem.

1.2 Background about the Project Idea

The idea grew out of the observation that small daily habits matter more than occasional health check-ups. Public health guidance commonly recommends roughly 7–9 hours of sleep, around 10,000 steps, regular exercise, adequate hydration and limited recreational screen time. These targets are well known, yet people rarely evaluate a whole day against all of them together.

At the same time, machine learning has become accessible through mature open-source libraries such as **scikit-learn**, and modern Python web frameworks such as **FastAPI** make it straightforward to serve a trained model as a REST API. Combining the two allows a lifestyle tracker to move beyond fixed if-else rules: a classifier can learn how several factors interact and can also report how confident it is in its decision.

A deliberate design decision was taken early: the project must **not** become a medical diagnosis tool. Publicly available "health" datasets are almost always clinical (blood pressure, glucose, disease labels). Instead, the project generates a **synthetic lifestyle dataset** with realistic value ranges and noisy labels, and keeps the ML prediction separate from a transparent, rule-based wellness score. This makes it very clear what the model is – and is not – responsible for, and makes the whole system easy to explain and defend.

1.3 Feasibility Study, Need and Significance

a) Technical Feasibility:

The project is technically feasible because every component uses mature, well-documented, open-source technology: Python, FastAPI, Pydantic, scikit-learn, pandas, NumPy, joblib, SQLAlchemy and SQLite on the backend, and HTML, CSS, JavaScript and Chart.js on the frontend. A Random Forest with 200 trees trains on 4,000 records in seconds on an ordinary laptop, and single-row inference is effectively instantaneous. No GPU, cloud service or external database server is required.

b) Economic Feasibility:

All tools and libraries used are free and open-source. The application runs entirely on the user's own machine, so there are no hosting, licensing, database or API costs. The only investment is development time, which makes the project highly economical for an academic setting.

c) Operational Feasibility:

The system is simple to operate. After a one-time setup (create a virtual environment, install the requirements, generate the dataset and train the model) the user starts one server with a single `uvicorn` command and opens one URL. FastAPI serves the frontend itself, so there is no separate frontend server. The SQLite database file is created automatically on first start-up.

d) Need and Significance:

Lifestyle-related problems develop gradually, and early awareness is the most effective prevention. A daily self-check that turns eleven easy numbers into a clear status, a score and a trend line encourages users to reflect on their habits consistently. Academically, the project is significant because it demonstrates the complete life-cycle of a machine-learning product – dataset creation, preprocessing, training, evaluation, model serialisation, API-based inference, persistence and visualisation – in a compact and explainable form.

1.4 Novelty of Project

The novelty of the **Daily Health & Wellness Monitor** lies less in any single algorithm and more in how the system is designed to be honest, explainable and cleanly separated:

- **Two different outputs, on purpose:** the wellness *category* comes from the trained Random Forest, while the 0–100 *score* comes from a transparent weighted formula. The classifier's probability is never disguised as a "score".
- **Noise-injected synthetic data:** Gaussian noise ($\sigma = 6$) is added to the reference score and about 6% of labels are flipped to a neighbouring class, so the model must learn a genuine, imperfect decision boundary rather than reverse-engineer arithmetic.
- **Deliberate "control" features:** age and weight are given to the model but excluded from the score formula; feature importances confirm that the model ranks them lowest.
- **Strict architectural boundaries:** the API layer never imports scikit-learn or opens the model file; only the ML prediction layer does. Likewise, all database access goes through a CRUD module.
- **Single-object model artefact:** preprocessing and model are saved together as one Pipeline along with the exact feature order, eliminating training/serving skew.

- **100% local and private:** no accounts, cloud services or third-party data sharing.

1.5 Technical Specification

The system is designed to run comfortably on an ordinary student laptop. The hardware and software required are listed below.

Hardware Requirements:

- Processor: Intel Core i3 / AMD Ryzen 3 or above
- RAM: Minimum 4 GB (8 GB recommended)
- Storage: About 1 GB free (Python virtual environment, libraries, dataset and model file)
- Display: 1366 × 768 resolution or higher
- Internet: Needed only once to install Python packages; the Chart.js library and web fonts are loaded from a CDN by the browser

Software Requirements:

- **Operating System:** Windows 10 / 11, Linux or macOS
- **Programming Language:** Python 3 (backend & ML), JavaScript (frontend)
- **Backend Framework:** FastAPI with Uvicorn ASGI server
- **Data Validation:** Pydantic
- **Machine Learning:** scikit-learn (RandomForestClassifier, StandardScaler, Pipeline), pandas, NumPy, joblib
- **Database:** SQLite accessed through SQLAlchemy ORM
- **Frontend:** HTML5, CSS3, vanilla JavaScript, Chart.js
- **Browser:** Google Chrome, Microsoft Edge or Mozilla Firefox (latest)
- **IDE:** Visual Studio Code
- **Version Control:** Git and GitHub
- **Package Manager:** pip with Python virtual environment (venv)

CHAPTER 2
AIM AND OBJECTIVE

Aim: To design and develop a locally-run web application, the **Daily Health & Wellness Monitor**, that accepts a user's daily lifestyle inputs, classifies the day into a wellness category using a trained Random Forest model, computes a transparent 0–100 wellness score, stores every entry and visualises the user's wellness trend over time.

To achieve the stated aim, the following objectives were defined and implemented during the project development process:

1. **To Build a Realistic Lifestyle Dataset**

- Generate a synthetic dataset of 4,000 records with eleven lifestyle features sampled from realistic distributions, and create noisy target labels that imitate imperfect, self-reported real-world data.

2. **To Train and Evaluate a Machine Learning Classifier**

- Train a RandomForestClassifier inside a scikit-learn Pipeline on a stratified 80/20 split and evaluate it using accuracy, macro precision, recall, F1-score, a classification report, a confusion matrix and feature importances.

3. **To Provide a Transparent Wellness Score**

- Implement a deterministic, weighted 0–100 scoring formula over nine lifestyle factors that is independent of the ML model and easy to explain.

4. **To Develop a Robust REST API**

- Expose the system through FastAPI endpoints (/health, /predict, /history) with strict server-side input validation using Pydantic and meaningful HTTP error codes.

5. **To Separate the ML Layer from the API Layer**

- Isolate model loading and inference in a dedicated prediction module so that the API never touches the model file or scikit-learn directly.

6. **To Persist Daily Records**

- Store every submission, together with its prediction and score, in a SQLite database using SQLAlchemy, with the database created automatically at start-up.

7. **To Create a Simple and Responsive Dashboard**

- Provide a single-page interface with grouped input fields, client-side validation, an instant result card and a Chart.js line chart of wellness score history.

8. **To Keep the System Local, Private and Responsible**

- Run fully on the user's machine without accounts or cloud services, and clearly communicate that the tool reflects lifestyle inputs only and is not a medical device.

CHAPTER 3
SCOPE OF PROJECT

The proposed system has a well-defined scope in terms of **functionality**, **technology** and **application**. It covers the following major areas:

3.1 Functional Scope

1. **Daily Data Entry:**
   An eleven-field form grouped into “Rest & activity”, “Body & vitals” and “Mind & screen”, with sensible default values and numeric validation in the browser.
2. **Wellness Classification:**
   Each submitted day is classified by the Random Forest into Good, Moderate or Needs Improvement, together with the predicted probability of every class.
3. **Wellness Score and Message:**
   A rule-based 0–100 score and a short human-readable message are returned with every prediction.
4. **History and Trend Visualisation:**
   Every entry is saved and the most recent records (default 60) are plotted as a line chart of wellness score over time.
5. **System Health and API Documentation:**
   A /health endpoint reports whether the trained model has loaded, and FastAPI automatically generates interactive documentation at /docs.

3.2 Technical Scope

1. **Technology Stack:**
   FastAPI and Pydantic for the API, scikit-learn, pandas, NumPy and joblib for machine learning, SQLAlchemy with SQLite for storage, and HTML/CSS/JavaScript with Chart.js for the user interface.
2. **Layered and Modular Design:**
   Separate modules for routing (main.py), validation (schemas.py), inference (ml_predictor.py), scoring (scoring.py), data access (crud.py) and database setup (database.py).
3. **Offline Training Workflow:**
   Dataset generation, training and evaluation are standalone scripts in the ml/ folder that produce the model file consumed by the live API.
4. **Portability:**
   Runs on Windows, Linux and macOS with only Python installed; no Docker or external database server is required.

3.3 Application Scope

1. **Personal Lifestyle Awareness:**
   Individuals, especially students and working professionals, can track daily habits and notice patterns such as poor sleep or excessive screen time.
2. **Academic Demonstration:**
   The project serves as a complete, explainable example of an end-to-end ML web application for teaching data generation, model evaluation and model serving.
3. **Foundation for Wellness Products:**
   The API-first design allows the same backend to be reused by a mobile app, a wearable integration or a corporate wellness dashboard in future.

3.4 Out-of-Scope Areas

To define project boundaries clearly, the following functionalities are considered out of scope for the current version:

- Medical diagnosis, disease prediction or any clinical decision support.
- Training on real patient or clinical datasets.
- User accounts, authentication and multi-user profiles (the current system is single-user and local).
- Automatic data import from wearables, fitness bands or phone sensors.
- Editing or deleting past entries and exporting history (listed as future work).
- Cloud deployment and mobile applications.

3.5 Summary of the Project Scope

In summary, the **Daily Health & Wellness Monitor** lies within the domain of applied machine learning and web development. It delivers a complete, local, single-user system that turns daily lifestyle inputs into an ML-based wellness category, an explainable score and a visual trend, while deliberately staying away from medical claims. Its modular design leaves clear room for future extensions such as multiple users, data export and real-world data collection.

CHAPTER 4
SYSTEM ANALYSIS & DESIGN

System Analysis and Design (SAD) is a critical phase of the Software Development Life Cycle (SDLC) that defines how the system will function internally and externally. It involves analysing user requirements, designing logical and physical architectures, and representing data and control flow through diagrams.

For the project **"Daily Health & Wellness Monitor"**, this phase defines the structure, behaviour and interaction between the frontend dashboard, the FastAPI backend, the ML prediction layer, the trained model and the SQLite database.

4.1 System Analysis

Existing System:

- Users rely on separate apps or memory for steps, sleep, water and mood; there is no combined daily view.
- Many trackers depend on a specific wearable or a cloud account and share personal data with servers.
- Scores shown by many apps are opaque – the user cannot tell how they are calculated.

Proposed System:

- One form captures eleven lifestyle values for the day.
- A trained Random Forest provides a category and class probabilities; a transparent formula provides the score.
- Every entry is stored locally in SQLite and visualised as a trend.
- Everything runs on the user's own machine.

Functional Requirements:

1. The system shall accept eleven numeric inputs and validate each against a defined range.
2. The system shall return the predicted status, confidence, class probabilities, wellness score and a message.
3. The system shall save each valid submission with a timestamp, prediction and score.
4. The system shall return recent history in chronological order and plot it.
5. The system shall report service and model status through a health endpoint.

Non-Functional Requirements:

- **Usability:** single page, grouped fields, default values, clear error messages.
- **Performance:** the model is loaded once (lazy singleton) and reused for every request.
- **Reliability:** missing model file results in a clear HTTP 503 message instead of a crash.
- **Maintainability:** strict separation of API, ML, scoring and database code.
- **Privacy:** no external data transfer; data stays in a local SQLite file.

4.2 System Architecture

The system follows a four-layer architecture. The browser talks only to the FastAPI backend; the backend delegates all ML work to ml_predictor.py and all database work to crud.py. Model training is a separate offline process that only produces the model file.

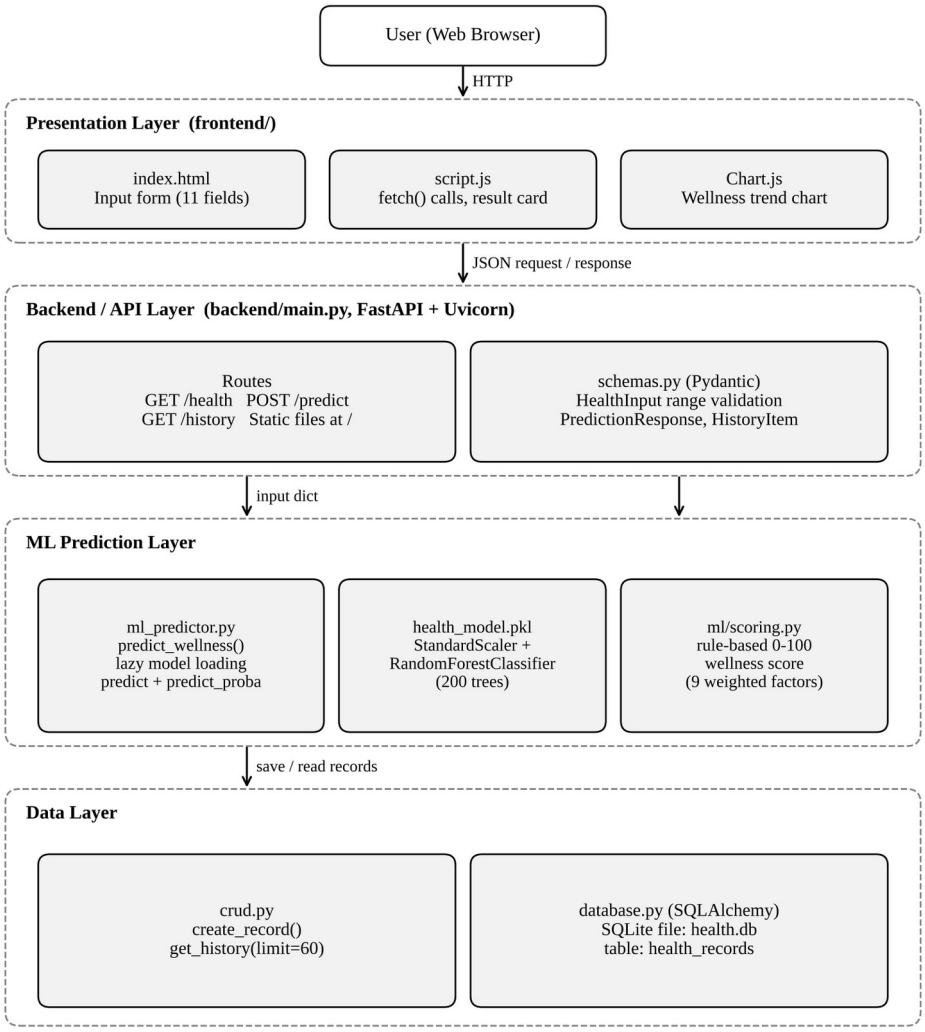


Figure 1: System Architecture of the Daily Health & Wellness Monitor

The offline training workflow that produces the model file used by the prediction layer is shown below.

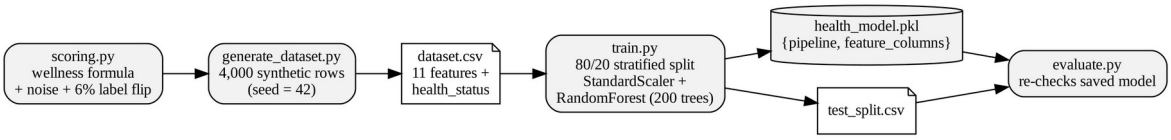


Figure 2: Offline Dataset Generation and Model Training Pipeline

4.3 Data Flow Diagrams (DFDs) of the Project

Data Flow Diagrams represent the logical flow of data within the system. They illustrate how information enters the system, which processes transform it and where it is stored. The DFDs below show how the Daily Health & Wellness Monitor handles daily inputs, predictions and history.

Level 0 DFD (Context Diagram)

This is the top-level view of the system, showing the whole application as a single process and its interaction with the external entity (the user) and the data store.

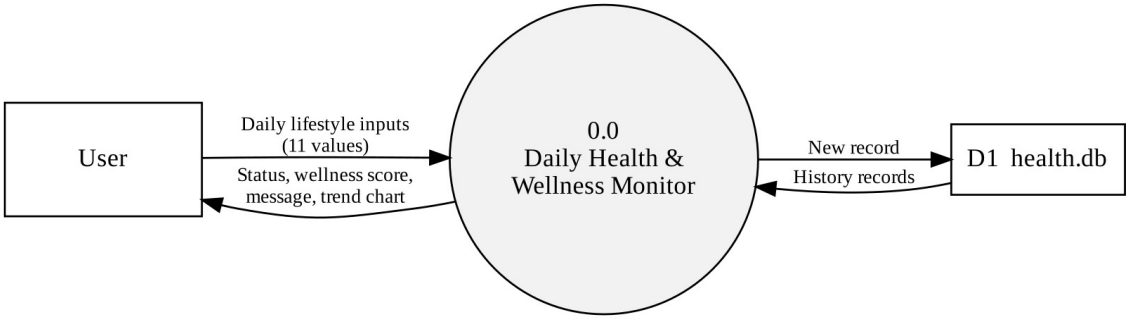


Figure 3: Level 0 DFD (Context Diagram)

Level 1 DFD (Expanded View)

This diagram decomposes the system into its five main processes: input capture and validation, wellness prediction, wellness score computation, record storage, and history retrieval with trend plotting.

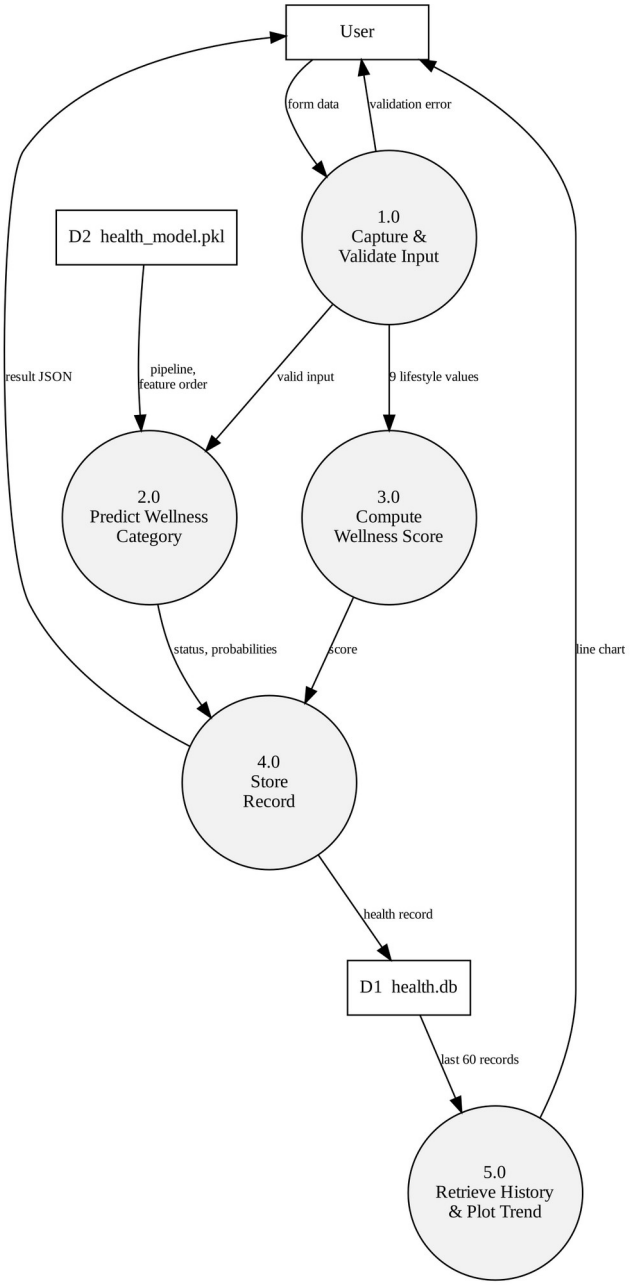


Figure 4: Level 1 DFD (Expanded View)

Level 2 DFD (Detailed Prediction Process 2.0)

This diagram expands process 2.0 to show how a validated input is turned into a prediction inside the ML prediction layer.

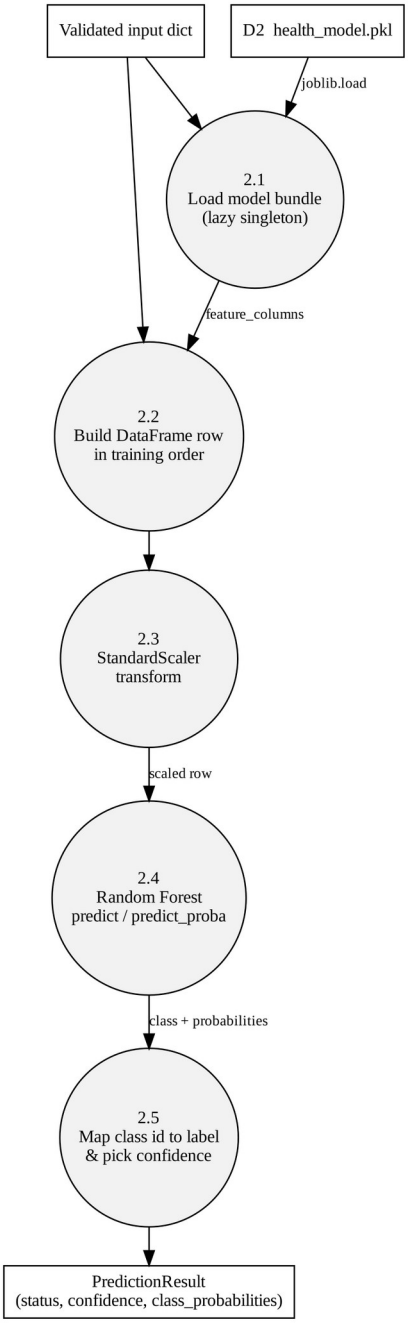


Figure 5: Level 2 DFD of the Prediction Process (2.0)

4.4 Flowchart

The flowchart shows the control flow of one daily analysis, including every validation and error path.

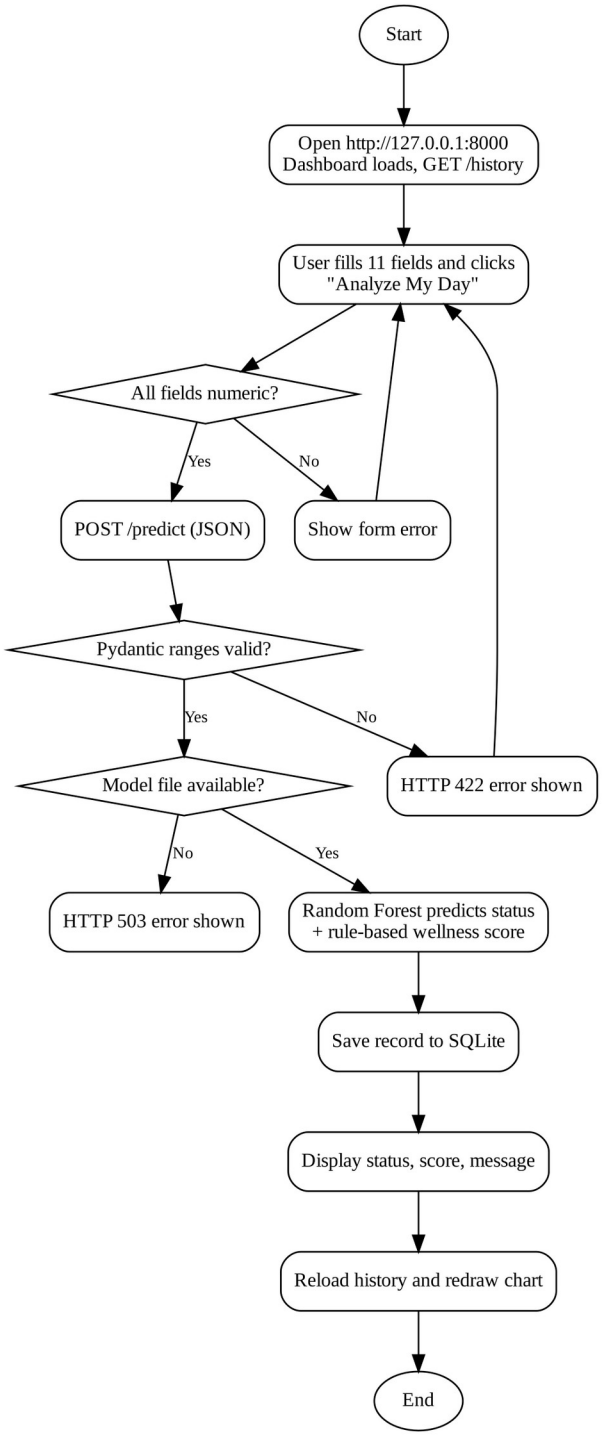


Figure 6: Flowchart of the Daily Entry Analysis

4.5 Use Case Diagram

Two actors interact with the system: the **User**, who enters data and views results, and the **Developer/Administrator**, who prepares the dataset, trains the model and inspects the API.

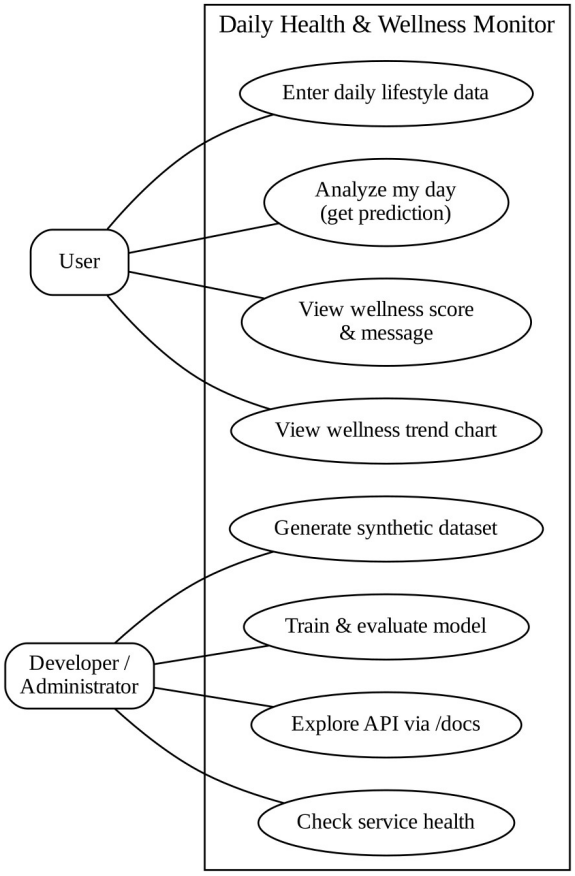


Figure 7: Use Case Diagram

4.6 Sequence Diagram

The sequence diagram shows the order of messages exchanged between components when the user clicks “Analyze My Day”, followed by the history refresh.

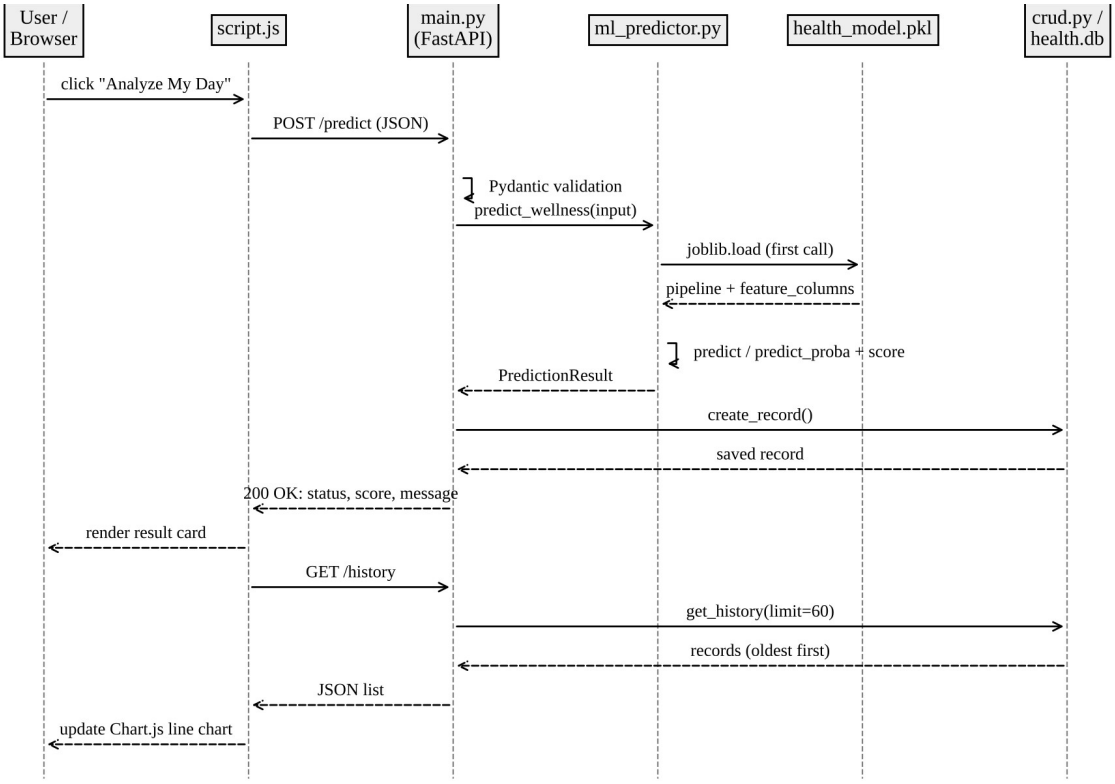


Figure 8: Sequence Diagram for POST /predict and GET /history

4.7 ER Diagram

The application stores data in a single SQLite table, health_records. Each row represents one daily entry together with the model's prediction and the computed wellness score.

HEALTH_RECORDS		
Field	**Type**	**Key / Constraint**
id	INTEGER	PK, indexed
date	DATETIME	default UTC now, indexed
age	INTEGER	NOT NULL
sleep_hours	FLOAT	NOT NULL
steps	INTEGER	NOT NULL
exercise_minutes	INTEGER	NOT NULL
water_intake	FLOAT	NOT NULL
stress_level	INTEGER	NOT NULL
heart_rate	INTEGER	NOT NULL
calories	INTEGER	NOT NULL
screen_time	FLOAT	NOT NULL
weight	FLOAT	NOT NULL
mood	INTEGER	NOT NULL
prediction	VARCHAR	NOT NULL
wellness_score	FLOAT	NOT NULL

Figure 9: ER Diagram of the health_records Table

4.8 Summary of System Design

The System Analysis and Design phase ensures that the Daily Health & Wellness Monitor is modular, explainable and easy to maintain:

- The **architecture diagram** shows the strict separation of presentation, API, ML and data layers.
- The **DFDs** define how data moves between processes and data stores.
- The **flowchart** and **sequence diagram** capture runtime behaviour, including error handling.
- The **use case** and **ER diagrams** define the actors and the persistent data model.

Together, these diagrams provide a complete blueprint for implementation.

CHAPTER 5
IMPLEMENTATION

Implementation Goal: To transform the design into a working application in which the trained model, API, database and dashboard operate together as one local system. The project folder is organised as follows:

Project folder structure

```
health-monitor/
|-- backend/
|   |-- main.py           # FastAPI app, routes, serves frontend
|   |-- ml_predictor.py   # ML prediction layer - loads .pkl, runs
inference
|   |-- schemas.py        # Pydantic request/response models
|   |-- database.py       # SQLAlchemy engine / session / table
|   |-- crud.py           # Database read/write functions
|   |-- requirements.txt
|   `-- model/health_model.pkl   # created by ml/train.py
|-- ml/
|   |-- scoring.py          # shared wellness-score formula
|   |-- generate_dataset.py # builds the synthetic dataset
|   |-- train.py            # trains + evaluates + saves the model
|   `-- evaluate.py         # re-evaluates the saved .pkl file
`-- frontend/  index.html | style.css | script.js
```

5.1 Tables – Fields and Data Types of the Database

The database file health.db is created automatically by database.py on start-up (Base.metadata.create_all). It contains one table, **health_records**:

Table 1: health_records table – fields and data types

Field	Data Type	Constraint	Description
id	Integer	Primary key, indexed	Unique record identifier
date	DateTime	Default: current UTC time, indexed	Time of submission
age	Integer	Not null	Age in years
sleep_hours	Float	Not null	Hours of sleep last night
steps	Integer	Not null	Daily step count
exercise_minutes	Integer	Not null	Minutes of exercise
water_intake	Float	Not null	Water intake in litres
stress_level	Integer	Not null	Self-rated stress (1–10)
heart_rate	Integer	Not null	Resting heart rate (bpm)
calories	Integer	Not null	Calories consumed
screen_time	Float	Not null	Screen time in hours
weight	Float	Not null	Weight in kg
mood	Integer	Not null	Self-rated mood (1–10)
prediction	String	Not null	Predicted category, e.g. “Good”
wellness_score	Float	Not null	Rule-based score (0–100)

The same eleven input fields are validated by the Pydantic model HealthInput before they reach the model or the database. The accepted ranges are:

Table 2: Server-side validation ranges (schemas.py)

Field	Allowed Range	Field	Allowed Range
age	10 – 100	heart_rate	30 – 220
sleep_hours	0 – 24	calories	0 – 10,000
steps	0 – 100,000	screen_time	0 – 24
exercise_minutes	0 – 600	weight	20 – 300
water_intake	0 – 15	mood	1 – 10
stress_level	1 – 10		

5.2 Used Algorithms / Approaches

a) Synthetic Data Generation:

generate_dataset.py samples 4,000 rows (random seed 42) from realistic distributions – for example sleep hours ~ Normal(6.4, 2.0) clipped to 0–14, steps ~ Normal(6200, 4600) clipped to 0–25,000, exercise minutes ~ Exponential(20) clipped to 0–150. The reference wellness score is computed for each row, Gaussian noise (standard deviation 6) is added, the noisy score is bucketed into three classes, and about 6% of labels are flipped to a neighbouring class. The resulting class balance is shown in Figure 10.

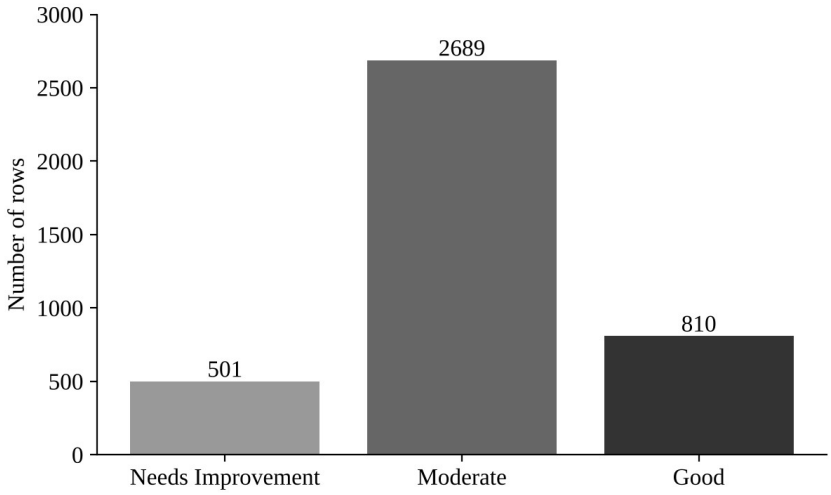


Figure 10: Class Distribution of the Synthetic Dataset (4,000 rows)

b) Rule-Based Wellness Score:

scoring.py converts nine factors into sub-scores between 0 and 1 using three helper shapes – “higher is better”, “lower is better” and “ideal band” – and combines them with weights that sum to 100. Age and weight are intentionally excluded. Scores ≥ 75 map to Good, 50–74.99 to Moderate and below 50 to Needs Improvement.

Table 3: Wellness score factors and weights (scoring.py)

Factor	Rule	Weight
Sleep hours	Ideal band 7–9 h (0 at 0 h and 14 h)	15
Steps	Higher is better, full at 10,000	15
Exercise minutes	Higher is better, full at 45 min	15
Stress level	Lower is better, full at ≤ 3, zero at 10	12
Resting heart rate	Ideal band 60–80 bpm (0 at 40 and 110)	10
Mood	Higher is better, full at 8	10
Water intake	Higher is better, full at 2.5 L	8
Calories	Ideal band 1,800–2,600 (0 at 1,000 and 4,000)	8
Screen time	Lower is better, full at ≤ 2 h, zero at 12 h	7

c) Random Forest Classifier:

Random Forest is an ensemble of decision trees, each trained on a bootstrap sample of the data and a random subset of features at every split; the final class is decided by averaging the trees' predicted probabilities. It handles non-linear interactions between features, is robust to noise, needs little tuning and provides feature importances. The configuration used is **n_estimators = 200, max_depth = 10, min_samples_leaf = 3, class_weight = "balanced", random_state = 42**. The balanced class weight compensates for the dominance of the Moderate class.

d) Preprocessing Pipeline:

A StandardScaler is placed before the classifier inside a scikit-learn Pipeline. Trees do not strictly need scaling, but bundling preprocessing with the model guarantees that the API applies exactly the same transformation as training and allows the model to be swapped for a scale-sensitive algorithm later. The data are split 80/20 with stratification on the target.

5.3 Implementation of Modules

Module 1 – Dataset Generation (ml/generate_dataset.py)

Creates dataset.csv with the eleven features, the clean reference score, the numeric target health_status (0, 1, 2) and its text label, and prints the class balance.

Module 2 – Model Training (ml/train.py)

Loads the dataset, drops rows with missing values, checks that the target only contains 0, 1 and 2, trains the pipeline, prints the evaluation metrics and feature importances, saves the held-out rows to test_split.csv, and stores the pipeline together with the feature order:

Code Snippet 1: Pipeline definition and model saving (train.py)

```
def build_pipeline() -> Pipeline:
    return Pipeline(steps=[
        ("scaler", StandardScaler()),
        ("model", RandomForestClassifier(
            n_estimators=200, max_depth=10, min_samples_leaf=3,
            random_state=42, class_weight="balanced")),
    ])

joblib.dump({"pipeline": pipeline, "feature_columns": FEATURE_COLUMNS},
            MODEL_OUTPUT_PATH)
```

Module 3 – Model Re-evaluation (ml/evaluate.py)

Loads the already-saved .pkl file and re-evaluates it on test_split.csv, proving that the file on disk is the same model that was reported during training.

Module 4 – ML Prediction Layer (backend/ml_predictor.py)

The only backend module that loads the model. The bundle is loaded lazily once and cached. The input row is built in exactly the training feature order, then predict() and predict_proba() are called, and the rule-based score is computed separately. A missing or corrupt model raises ModelNotLoadedError.

Code Snippet 2: Core inference logic (ml_predictor.py)

```
row = pd.DataFrame([[input_data[col] for col in feature_columns]],
                   columns=feature_columns)
predicted_class = int(pipeline.predict(row)[0])
probabilities   = pipeline.predict_proba(row)[0]
confidence      = round(float(probabilities[predicted_class]), 4)
wellness_score  = compute_wellness_score(...)   # rule-based, independent
```

Module 5 – Backend API (backend/main.py)

A thin FastAPI application: it validates the request with Pydantic, calls predict_wellness(), saves the record through crud.py and returns JSON. It converts ModelNotLoadedError into HTTP 503 and ValueError into HTTP 422, enables CORS, creates the database on start-up and finally mounts the frontend folder at “/”.

Table 4: REST API endpoints

Method	Endpoint	Description
GET	/health	Liveness check; reports whether the model loaded
POST	/predict	Submit today’s data and receive a prediction
GET	/history?limit=60	Recent saved records, oldest first
GET	/docs	Auto-generated interactive API documentation

Module 6 – Database and CRUD (backend/database.py, backend/crud.py)

database.py defines the SQLAlchemy engine (with check_same_thread = False for FastAPI), the session factory, the HealthRecord model and a get_db() dependency that always closes the session. crud.py provides create_record() and get_history(), which returns the latest records reversed into chronological order for plotting.

Module 7 – Frontend Dashboard (frontend/)

index.html contains the grouped form, the result card and a canvas for the chart. script.js collects and type-converts the inputs, shows a form error for empty fields, disables the button

while “Analyzing…”, sends POST /predict, renders the status (colour-coded), rounded score and message, and then reloads GET /history to update the Chart.js line chart (y-axis fixed from 0 to 100). style.css provides a clean card-based responsive layout, and a footer disclaimer states that the tool is not medical advice.

5.4 Execution of the Project

The project is run locally with the following commands (Windows):

Setup and run commands

```
python -m venv venv
venv\Scripts\activate
pip install -r backend\requirements.txt
python ml\generate_dataset.py
python ml\train.py
uvicorn backend.main:app --reload
# open http://127.0.0.1:8000   (API docs: http://127.0.0.1:8000/docs)
```

CHAPTER 6
RESULTS / OUTPUTS & TESTING

6.1 All User Interfaces and Output Screens

The Daily Health & Wellness Monitor provides a single dashboard page served by FastAPI at http://127.0.0.1:8000. The main screens and outputs are described below, followed by screenshots of the running application.

1. **Dashboard / Home Page:**
   Title “Daily Health & Wellness Monitor”, the entry form on the left, and the result card and trend chart on the right. Before the first analysis the result card shows a hint and the chart shows “No entries yet”.
2. **Daily Entry Form:**
   Eleven fields in three groups – Rest & activity, Body & vitals, Mind & screen – with the “Analyze My Day” button.
3. **Result Card:**
   Colour-coded status badge (Good / Moderate / Needs Improvement), the wellness score out of 100 and a short message.
4. **Wellness Over Time Chart:**
   A Chart.js line chart of the wellness score of saved entries.
5. **Error Messages:**
   Inline error text for empty inputs or values rejected by the server.

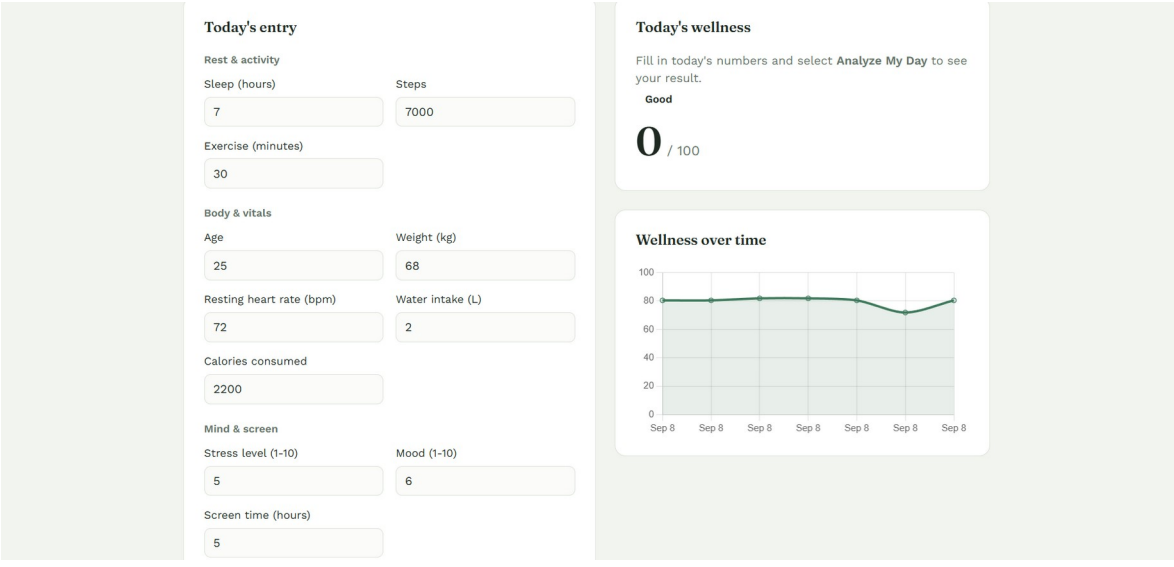


Figure 11: Dashboard Home Page (Before Analysis)

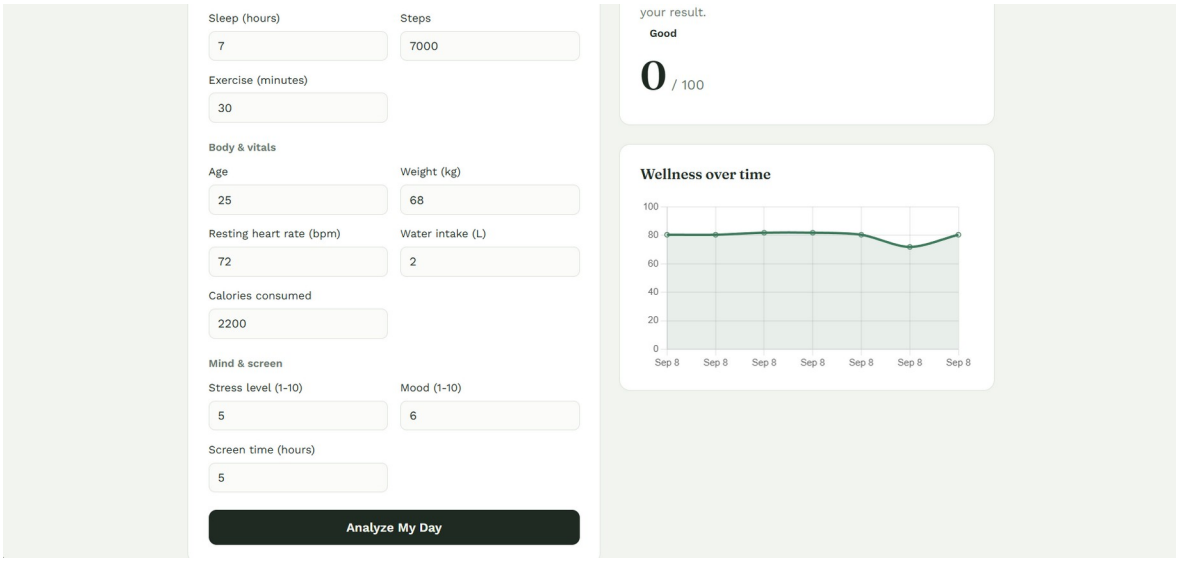


Figure 12: Daily Entry Form Filled with Today's Values

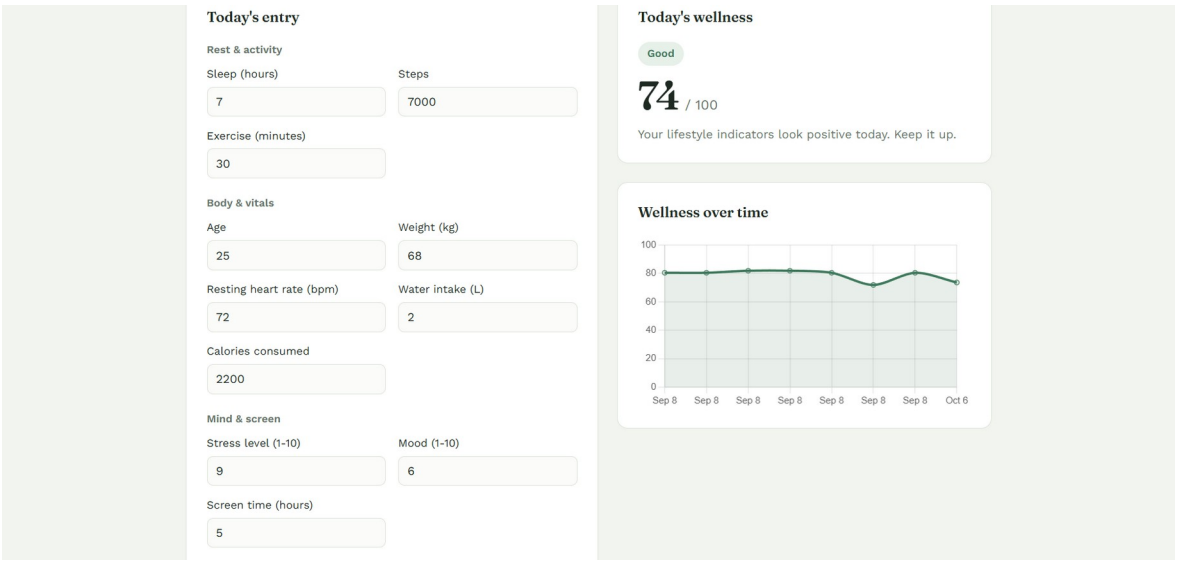


Figure 13: Result Card Showing "Good" Status (Wellness Score 74/100)

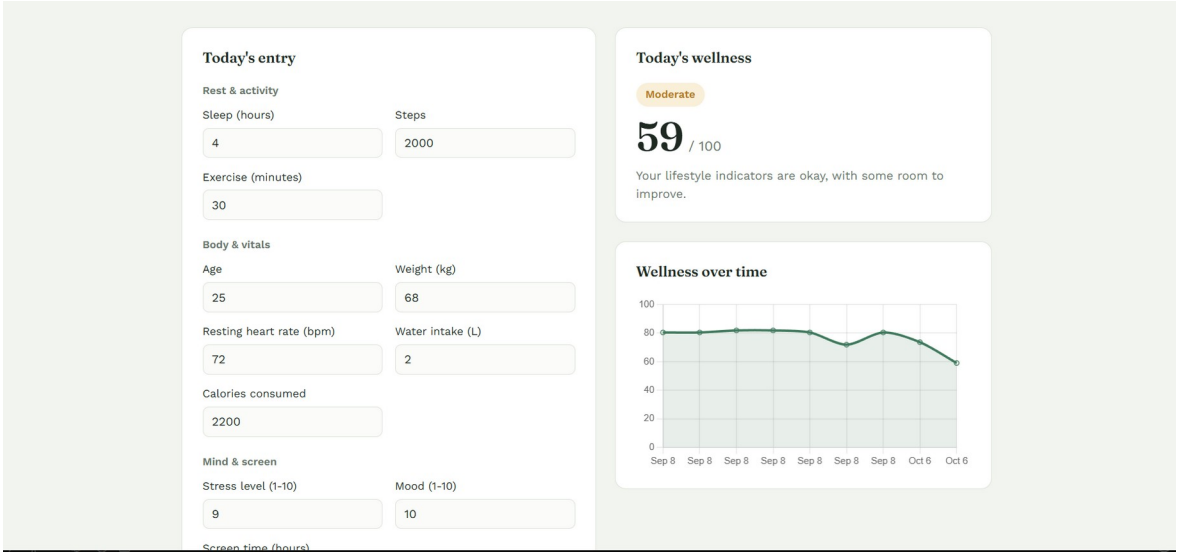


Figure 14: Result Card Showing "Moderate" Status (Wellness Score 59/100)

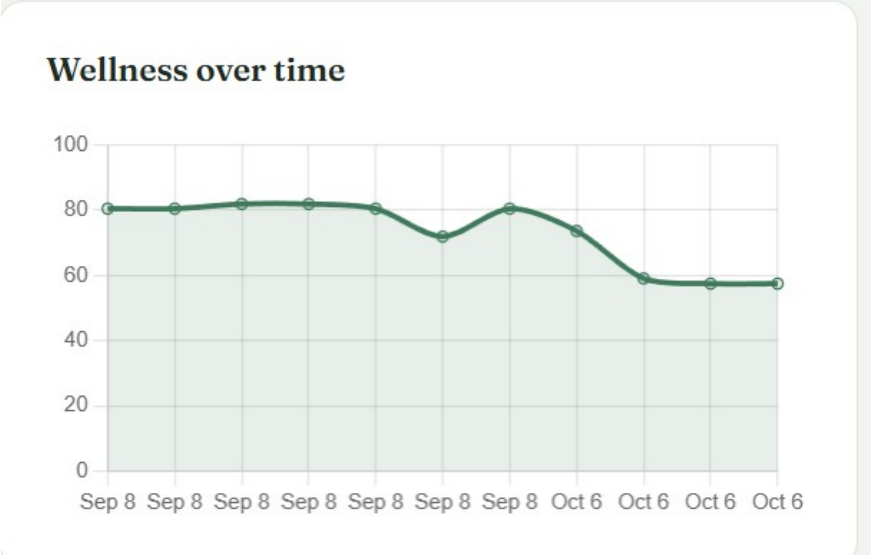


Figure 15: Wellness Over Time Trend Chart

6.2 API Outputs

Because the backend is built with FastAPI, an interactive Swagger UI is generated automatically at /docs. It was used to test each endpoint directly, and every successful prediction is stored as a new row in the health_records table of the SQLite database.

A typical response of POST /predict has the following structure:

Sample JSON response of POST /predict

```
{
  "status": "Good",
  "wellness_score": 91.68,
  "confidence": 0.9686,
  "class_probabilities": {"Needs Improvement": 0.0044,
                          "Moderate": 0.027, "Good": 0.9686},
  "message": "Your lifestyle indicators look positive today. Keep it up."
}
```

6.3 Model Evaluation Results

The model was trained and evaluated by running the project's own scripts (generate_dataset.py, train.py and evaluate.py) with the fixed random seed 42. The held-out test set contains 800 rows (20% of 4,000). The results below are the actual printed output of train.py; minor differences may occur with other library versions (scikit-learn 1.9.1, pandas 3.0.5 and NumPy 2.5.3 were used here).

Table 5: Overall evaluation metrics on the test set

Metric	Value
Accuracy	0.6787 (67.87%)
Precision (macro)	0.5955
Recall (macro)	0.6507
F1-score (macro)	0.6160
Weighted F1-score	0.69

Table 6: Class-wise classification report

Class	Precision	Recall	F1-score	Support
Needs Improvement	0.41	0.57	0.48	100
Moderate	0.80	0.70	0.75	538
Good	0.58	0.69	0.63	162

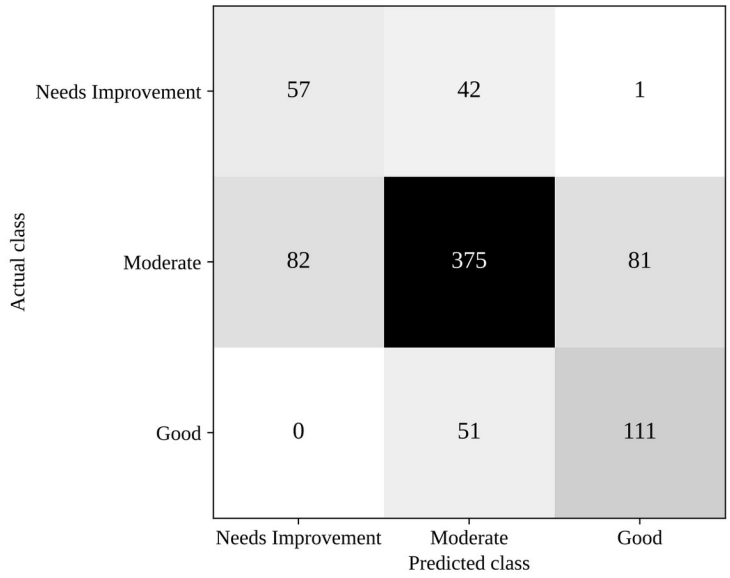


Figure 16: Confusion Matrix of the Random Forest on the Test Set

The confusion matrix shows that almost all errors occur between **neighbouring** classes: only one “Needs Improvement” day was predicted as “Good”, and no “Good” day was predicted as “Needs Improvement”. This is expected, because the labels were intentionally made noisy near the class boundaries (Gaussian noise plus 6% neighbouring-class label flips), which places a natural ceiling on achievable accuracy.

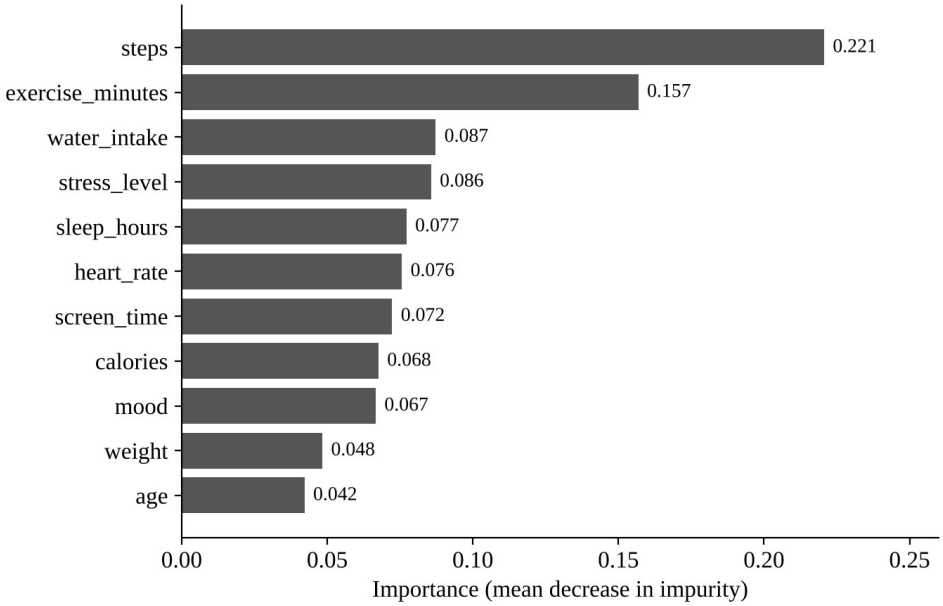


Figure 17: Feature Importances of the Trained Random Forest

Steps (0.221) and exercise minutes (0.157) are the most influential features, while **weight (0.048) and age (0.042) are ranked lowest** – exactly as intended, since these two features were not used in the score formula that generated the labels. This confirms that the model has learned meaningful relationships rather than spurious ones.

Sample Predictions:

The trained model was queried directly through predict_wellness() with the following inputs:

Table 7: Sample predictions from the trained model

Input profile (key values)	Predicted Status	Confidence	Wellness Score
Sleep 7.5 h, 9,000 steps, 40 min exercise, stress 4, mood 7, screen 3.5 h	Good	0.9686	91.68
Sleep 7 h, 7,000 steps, 30 min exercise, stress 5, mood 6, screen 5 h (form defaults)	Good	0.7702	80.51
Sleep 6 h, 5,000 steps, 15 min exercise, stress 6, mood 6, screen 6 h	Moderate	0.5693	66.36
Sleep 4 h, 1,500 steps, no exercise, stress 9, mood 3, screen 11 h	Needs Improvement	0.8780	26.51

6.4 Design and Test Steps / Criteria

Testing was carried out to verify that every module behaves as expected and that the layers work correctly together.

1. **Data and Model Verification:**
   Checked class balance of the generated data, missing-value handling and target validation in train.py; confirmed with evaluate.py that the saved .pkl reproduces the training metrics.
2. **ML Layer Testing:**
   Called predict_wellness() directly with good, moderate and poor profiles, with a missing feature and with the model file removed.
3. **API Testing:**
   Exercised /health, /predict and /history through the Swagger UI at /docs with valid and out-of-range inputs.
4. **UI Testing:**
   Submitted entries from the dashboard, checked the result card, error messages and chart refresh in Chrome and Edge.
5. **Acceptance Criteria:**
   Correct status/score for each input, correct HTTP status codes, every valid submission saved and plotted, and no unhandled errors.

6.5 Testing Process

The testing followed a bottom-up approach – unit level (scoring and prediction functions), integration level (API + ML layer + database) and system level (complete flow from the browser). The test cases are listed below together with the result obtained for each case.

Table 8: Test cases

ID	Test Case	Expected Result	Observed Result
T1	Ideal inputs to scoring formula	Score = 100	Score = 100.0
T2	Worst inputs to scoring formula	Score = 0	Score = 0.0
T3	Healthy profile to predict_wellness()	Status “Good”	Good (0.9686)
T4	Poor profile to predict_wellness()	Status “Needs Improvement”	Needs Improvement (0.878)
T5	Input with missing feature	ValueError: Missing required feature	ValueError raised
T6	Model file removed	ModelNotLoadedError; is_model_ready() = False	As expected
T7	GET /health with model present	{"status":"ok","model_loaded" :true}	HTTP 200, model_loaded = true – Pass
T8	POST /predict with valid JSON	HTTP 200 with status, score, probabilities	HTTP 200, Good, score 91.68 – Pass
T9	POST /predict with stress_level = 15	HTTP 422 validation error	HTTP 422 – Pass
T10	POST /predict before training model	HTTP 503 with instructions	HTTP 503 – Pass
T11	GET /history after 3 submissions	3 records, oldest first	HTTP 200, 3 records, oldest first – Pass
T12	Empty field submitted from dashboard	Inline form error, no request sent	Error: Please enter a value for “sleep hours”. – Pass
T13	Valid submission from dashboard	Result card shown, chart updated	Good, 74/100, chart updated – Pass

Observations:

- The trained model reproduces identical metrics when reloaded from disk by evaluate.py.
- The wellness score and the ML category usually agree, but can differ near class boundaries because the model was trained on noisy labels – this is an intentional and explainable property of the design.
- Most misclassifications are between adjacent categories; extreme mistakes (Good ↔ Needs Improvement) are very rare (1 out of 800 test rows).
- The “Needs Improvement” class has the lowest precision (0.41) because it is the smallest class (501 of 4,000 rows).

Limitations:

- The model is trained on synthetic data, so its accuracy reflects the synthetic labelling process, not real-world validation.
- Inputs such as stress and mood are self-reported and subjective.

- Single-user, local-only system without authentication, editing or export of entries.
- Chart.js is loaded from a CDN, so the chart needs an internet connection in the browser.

CHAPTER 7
CONCLUSION AND FUTURE WORK

7.1 Conclusion

The project **"Daily Health & Wellness Monitor"** successfully demonstrates how everyday lifestyle data can be turned into a meaningful, easy-to-understand wellness indicator using machine learning. The user enters eleven simple values, and the system returns an ML-based category with class probabilities, a transparent 0–100 wellness score, a short message and a historical trend chart.

The project fulfils all its defined objectives. A realistic, noise-injected synthetic dataset was created; a Random Forest pipeline was trained and evaluated (accuracy 67.87%, macro F1 0.616 on 800 test rows) with feature importances that behave exactly as the design predicts; and the model was served through a clean FastAPI backend with Pydantic validation, SQLite persistence and a responsive dashboard.

Key Achievements:

- End-to-end ML workflow: data generation, training, evaluation, serialisation and live inference.
- Clear separation between the ML prediction and the rule-based wellness score.
- Layered architecture where the API never touches the model file or the database directly.
- Automatic database creation, robust error handling (HTTP 422 / 503) and auto-generated API docs.
- Fully local, private and free-to-run system with a responsible "not a medical device" stance.

Overall, the project is a successful implementation of an explainable, end-to-end machine-learning web application and provides a solid base for further development.

7.2 Future Work

While the current version meets its core objectives, several enhancements are possible:

1. **Edit and Delete Entries:**
   Allow the user to correct or remove a past record.
2. **Export History:**
   Download the stored history as a CSV file for personal analysis.
3. **Streaks and Weekly Summary:**
   Add weekly averages, streaks of good days and simple insights.
4. **Multiple User Profiles:**
   Introduce user accounts and authentication so that several people can use one installation.
5. **Automated Testing:**
   Add unit tests for ml_predictor.py and the API routes and run them in continuous integration.

6. **Real-World Data:**
   Collect consented, anonymised real lifestyle data to retrain and validate the model.
7. **Wearable Integration:**
   Import steps, sleep and heart rate automatically from fitness bands or phone health APIs.
8. **Explainability:**
   Show which factors pulled the day’s result up or down, using the per-factor sub-scores.

CHAPTER 8
REFERENCES

1. L. Breiman, “Random Forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.
2. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
3. scikit-learn Documentation – RandomForestClassifier, Pipeline and StandardScaler. https://scikit-learn.org/stable/
4. FastAPI Documentation. https://fastapi.tiangolo.com/
5. Pydantic Documentation. https://docs.pydantic.dev/
6. Uvicorn Documentation. https://www.uvicorn.org/
7. SQLAlchemy Documentation (ORM). https://docs.sqlalchemy.org/
8. SQLite Documentation. https://www.sqlite.org/docs.html
9. pandas Documentation. https://pandas.pydata.org/docs/
10. NumPy Documentation – Random Generator. https://numpy.org/doc/stable/
11. joblib Documentation – Persistence. https://joblib.readthedocs.io/
12. Chart.js Documentation. https://www.chartjs.org/docs/latest/
13. MDN Web Docs – Fetch API and HTML Forms. https://developer.mozilla.org/
14. World Health Organization, “WHO Guidelines on Physical Activity and Sedentary Behaviour,” Geneva, 2020.
15. M. Hirshkowitz et al., “National Sleep Foundation’s sleep time duration recommendations: methodology and results summary,” Sleep Health, vol. 1, no. 1, pp. 40–43, 2015.